

**UNIVERSIDADE PAULISTA**  
**ANÁLISE E DESENVOLVIMENTO DE SISTEMAS**

**ADRIEL THIAGO VENANCIO PEREIRA - F360695 (DS4P17)**

**AMANDA ALMEIDA RIBEIRO - H6389I8 (DS4Q17)**

**ANA JÚLIA SAMPAIO ALMEIDA - H471559 (DS4P17)**

**JEAN CAMARGO DE OLIVEIRA - R8693H0 (DS4P17)**

**MARCOS PAULO DE OLIVEIRA DIAS - F3651F1 (DS3P17)**

**PEDRO HENRIQUE BELEM SENA - R954GF0 (DS3P17)**

**DESENVOLVIMENTO DE UM SISTEMA WEB INTEGRADO DE GESTÃO E**  
**RESERVA DE ÁREAS COMUNS EM CONDOMÍNIO**

**SOROCABA – SP**

**2026**

**ADRIEL THIAGO VENANCIO PEREIRA - F360695 (DS4P17)**

**AMANDA ALMEIDA RIBEIRO - H6389I8 (DS4Q17)**

**ANA JÚLIA SAMPAIO ALMEIDA - H471559 (DS4P17)**

**JEAN CAMARGO DE OLIVEIRA - R8693H0 (DS4P17)**

**MARCOS PAULO DE OLIVEIRA DIAS - F3651F1 (DS3P17)**

**PEDRO HENRIQUE BELEM SENA - R954GF0 (DS3P17)**

**DESENVOLVIMENTO DE UM SISTEMA WEB INTEGRADO DE GESTÃO E  
RESERVA DE ÁREAS COMUNS EM CONDOMÍNIO**

Trabalho acadêmico da disciplina de Banco de Dados do curso de Análise e Desenvolvimento de Sistemas apresentado à Universidade Paulista – UNIP.

**SOROCABA – SP**

**2026**

## RESUMO

Este trabalho apresenta o desenvolvimento de um sistema web integrado a um banco de dados relacional para o gerenciamento de moradores, áreas comuns e reservas do Condomínio Sun Lake. A aplicação foi desenvolvida com Microsoft SQL Server, Python, HTML5, CSS3 e JavaScript, permitindo centralizar informações e organizar o processo de reservas. O sistema possui diferentes níveis de acesso para Administrador e Morador, possibilitando operações como cadastro e consulta de moradores, gerenciamento de áreas comuns, criação e cancelamento de reservas e consulta de registros. Foram implementadas regras de negócio para garantir a integridade das informações, incluindo o controle de áreas ativas, validação de datas e horários, prevenção de conflitos de reservas e manutenção do histórico de registros cancelados. O projeto também contempla a modelagem relacional do banco de dados, definição de chaves e restrições de integridade, integração entre back-end e banco de dados e realização de testes com diferentes cenários. Dessa forma, a aplicação demonstra a utilização prática de conceitos de banco de dados e desenvolvimento de sistemas na resolução de uma necessidade de gestão condominial.

**Palavras-chave:** Banco de dados. Sistema web. Reservas. Condomínio. SQL Server.

## **ABSTRACT**

This paper presents the development of a web-based system integrated with a relational database to manage residents, common areas, and reservations for the Sun Lake Condominium. The application was developed using Microsoft SQL Server, Python, HTML5, CSS3, and JavaScript, enabling the centralization of information and the organization of the reservation process. The system features distinct access levels for administrators and residents, allowing for operations such as resident registration and lookup, common area management, reservation creation and cancellation, and record querying. Business rules were implemented to ensure data integrity, including active area control, date and time validation, reservation conflict prevention, and the maintenance of a history of cancelled records. The project also covers relational database modeling, the definition of keys and integrity constraints, back-end and database integration, and testing across various scenarios. Thus, the application demonstrates the practical application of database and system development concepts in addressing a condominium management need.

**Keywords:** Database. Web system. Reservations. Condominium. SQL Server.

## SUMÁRIO

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>1 INTRODUÇÃO .....</b>                                   | <b>6</b>  |
| <b>2 CONTEXTUALIZAÇÃO .....</b>                             | <b>7</b>  |
| 2.1 Justificativa .....                                     | 7         |
| 2.2 Objetivos .....                                         | 7         |
| 2.3 Especificação tecnológica .....                         | 8         |
| <b>3 ESCOPO DO SISTEMA E PERFIS DE ACESSO .....</b>         | <b>9</b>  |
| 3.1 Administrador .....                                     | 9         |
| 3.2 Morador .....                                           | 9         |
| <b>4 REGRAS DE NEGÓCIO .....</b>                            | <b>10</b> |
| <b>5 MODELAGEM DO BANCO DE DADOS .....</b>                  | <b>12</b> |
| 5.1 Entidades e atributos .....                             | 12        |
| 5.2 Relacionamentos e cardinalidades .....                  | 12        |
| 5.3 Integridade referencial e restrições .....              | 14        |
| 5.3.1 Política de preservação dos registros .....           | 14        |
| <b>6 ARQUITETURA DE SOFTWARE E BACK-END .....</b>           | <b>15</b> |
| 6.1 Atribuições dos módulos .....                           | 15        |
| <b>7 IMPLEMENTAÇÃO E ACESSO A DADOS .....</b>               | <b>16</b> |
| 7.1 Autenticação e gerenciamento de moradores .....         | 16        |
| 7.2 Lógica de conflito de horários .....                    | 16        |
| <b>8 CAMADA WEB E INTERFACE .....</b>                       | <b>17</b> |
| 8.1 Rotas da aplicação .....                                | 17        |
| 8.2 Interface e lógica do cliente .....                     | 17        |
| <b>9 TESTES E DADOS DE TESTE .....</b>                      | <b>18</b> |
| <b>10 INSTALAÇÃO E EXECUÇÃO .....</b>                       | <b>19</b> |
| <b>11 CONSIDERAÇÕES FINAIS .....</b>                        | <b>24</b> |
| <b>APÊNDICE A – DDL DO BANCO DE DADOS .....</b>             | <b>25</b> |
| <b>APÊNDICE B – DADOS DE TESTE .....</b>                    | <b>27</b> |
| <b>APÊNDICE C – EVIDÊNCIAS VISUAIS DO BANCO .....</b>       | <b>29</b> |
| <b>APÊNDICE d – EVIDÊNCIAS VISUAIS DO SISTEMA WEB .....</b> | <b>36</b> |

## 1 INTRODUÇÃO

A utilização de sistemas informatizados tem contribuído para a organização e otimização de processos em diferentes ambientes, incluindo a administração de condomínios residenciais. Entre as atividades que podem ser aprimoradas por meio da tecnologia está o gerenciamento das áreas comuns, que envolve o controle de moradores, espaços disponíveis, horários e reservas. Quando essas atividades são realizadas de forma manual ou por meio de ferramentas descentralizadas, podem ocorrer conflitos de horários, dificuldades na consulta das informações e falhas no controle dos registros.

Diante desse contexto, este trabalho apresenta o desenvolvimento de um sistema web integrado a um banco de dados relacional para o gerenciamento de moradores, áreas comuns e reservas do Condomínio Sun Lake. A proposta consiste em centralizar as informações em uma única aplicação, permitindo que diferentes operações sejam realizadas de maneira organizada e controlada, de acordo com o perfil de acesso de cada usuário.

O sistema foi desenvolvido utilizando Microsoft SQL Server como sistema gerenciador de banco de dados, Python no desenvolvimento do back-end e HTML5, CSS3 e JavaScript na construção da interface web. A aplicação também utiliza regras de negócio responsáveis por controlar situações como áreas inativas, datas inválidas, horários fora do período permitido e conflitos entre reservas. Dessa forma, busca-se garantir maior consistência e integridade das informações armazenadas.

Além da implementação das funcionalidades, o projeto contempla a modelagem do banco de dados, definição dos relacionamentos entre as entidades, aplicação de chaves e restrições de integridade e realização de testes com diferentes cenários. Entre eles, destaca-se a utilização de reservas com horários consecutivos, demonstrando que uma reserva pode terminar exatamente quando outra se inicia, sem que isso seja considerado um conflito.

Assim, o desenvolvimento do sistema possibilita aplicar, de forma prática, conceitos relacionados a banco de dados, desenvolvimento web, regras de negócio e integração entre diferentes camadas de uma aplicação, apresentando uma solução voltada à organização e ao controle das reservas de áreas comuns em um ambiente condominial.

## **2 CONTEXTUALIZAÇÃO**

A gestão de áreas comuns em condomínios envolve atividades como cadastro de moradores, controle de espaços disponíveis e organização das reservas. Quando essas atividades são realizadas de maneira manual ou descentralizada, podem ocorrer conflitos de horários, dificuldades de consulta e perda de informações.

O projeto do Condomínio Sun Lake propõe uma aplicação web integrada a um banco de dados relacional para centralizar essas operações. A solução permite controlar moradores, áreas comuns e reservas, aplicando regras de negócio para reduzir conflitos e manter a integridade dos dados.

### **2.1 Justificativa**

A informatização do processo de reservas facilita a consulta de disponibilidade e reduz a ocorrência de registros incompatíveis. No contexto do Condomínio Sun Lake, a aplicação centraliza informações e disponibiliza funcionalidades de acordo com os perfis de Administrador e Morador.

### **2.2 Objetivos**

O objetivo geral é desenvolver uma aplicação web integrada a um banco de dados relacional para auxiliar a gestão de moradores, áreas comuns e reservas do Condomínio Sun Lake.

- Permitir cadastro, consulta e alteração de moradores.
- Permitir cadastro e consulta de áreas comuns e controle de seus status.
- Permitir criação, consulta e cancelamento de reservas.
- Impedir reservas em áreas inativas.
- Impedir reservas com datas passadas e horários inválidos.
- Validar conflitos de horário para uma mesma área.
- Diferenciar as funcionalidades disponíveis para Administrador e Morador.
- Manter os dados persistidos no SQL Server.

## **2.3 Especificação tecnológica**

O sistema utiliza Microsoft SQL Server como SGBD. O back-end foi desenvolvido em Python, utilizando módulos nativos e pyodbc para conexão direta com o SQL Server. A camada web utiliza HTML5, CSS3 e JavaScript puro, com Bootstrap Icons. O servidor foi implementado com http.server, sem frameworks de back-end e sem ORM.

### **3 ESCOPO DO SISTEMA E PERFIS DE ACESSO**

O escopo compreende o gerenciamento de usuários, moradores, áreas comuns e reservas.

#### **3.1 Administrador**

- Realizar login.
- Cadastrar, consultar e alterar moradores.
- Cadastrar e consultar áreas comuns.
- Ativar ou inativar áreas comuns.
- Consultar reservas.
- Cancelar reservas administrativamente.

#### **3.2 Morador**

- Realizar login.
- Consultar áreas comuns disponíveis.
- Criar reservas de áreas ativas.
- Consultar suas próprias reservas.
- Cancelar suas próprias reservas.

#### 4 REGRAS DE NEGÓCIO

A seguir tem-se a Tabela 1, que apresenta as regras de negócio estabelecidas para o funcionamento do sistema, definindo as condições que devem ser seguidas durante o cadastro de usuários e moradores, bem como no processo de criação e gerenciamento das reservas das áreas comuns. As regras especificam critérios relacionados aos cadastros, à relação entre usuários e moradores, à disponibilidade das áreas, aos horários permitidos, ao controle de conflitos entre reservas e ao registro de cancelamentos. Dessa forma, a Tabela 1 reúne as principais condições utilizadas para garantir o funcionamento adequado do sistema.

Tabela 1 – Regras de negócio

| Regra | Descrição                                                                                               |
|-------|---------------------------------------------------------------------------------------------------------|
| RN01  | E-mail do usuário deve ser único.                                                                       |
| RN02  | CPF do morador deve ser único.                                                                          |
| RN03  | Todo usuário cadastrado com o tipo MORADOR deve possuir um cadastro correspondente na entidade MORADOR. |
| RN04  | Um morador pode possuir várias reservas.                                                                |
| RN05  | Cada reserva pertence a um morador e a uma área.                                                        |
| RN06  | Não é permitida reserva para área INATIVA.                                                              |
| RN07  | A data da reserva não pode ser anterior à data atual.                                                   |
| RN08  | O horário final deve ser posterior ao inicial.                                                          |
| RN09  | As reservas devem ocorrer entre 08:00 e 23:00.                                                          |
| RN10  | Não é permitido sobrepor reservas ATIVAS para a mesma área e data.                                      |

|      |                                                                                                                                                                                                                                                                                                                                                           |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RN11 | Horários consecutivos são permitidos quando uma reserva termina exatamente quando outra começa.                                                                                                                                                                                                                                                           |
| RN12 | Reservas canceladas permanecem registradas com status CANCELADA.                                                                                                                                                                                                                                                                                          |
| RN13 | O Morador consulta/cancela suas próprias reservas; o Administrador possui visão administrativa.                                                                                                                                                                                                                                                           |
| RN14 | Os registros que não estiverem mais em utilização devem ser inativados ou cancelados, conforme o caso, em vez de serem excluídos fisicamente quando sua remoção puder comprometer o histórico ou os relacionamentos do banco de dados. Dessa forma, os dados permanecem armazenados e podem ser consultados posteriormente por meio de filtros de status. |

Fonte: Autoria própria

## 5 MODELAGEM DO BANCO DE DADOS

O banco trabalhonp1 foi modelado de forma relacional. As entidades principais são USUARIO, MORADOR, AREA\_COMUM e RESERVA. A estrutura utiliza chaves primárias, chaves estrangeiras e restrições de integridade.

### 5.1 Entidades e atributos

USUARIO: id\_usuario, email, senha, tipo\_usuario e status. MORADOR: id\_morador, id\_usuario, nome, cpf, telefone, bloco e apartamento. AREA\_COMUM: id\_area, nome, descricao, capacidade e status. RESERVA: id\_reserva, id\_morador, id\_area, data\_reserva, hora\_inicio, hora\_fim e status.

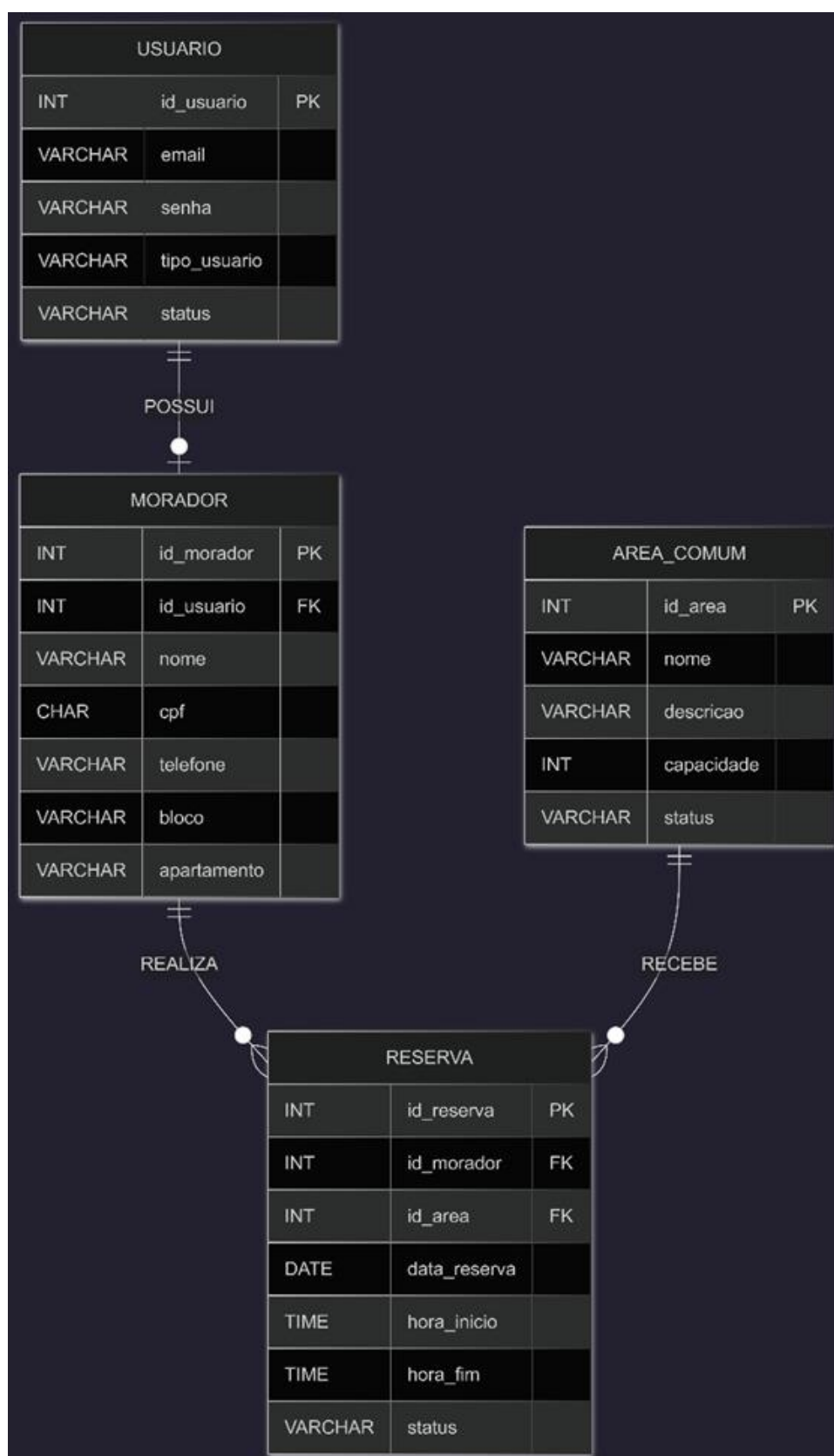
### 5.2 Relacionamentos e cardinalidades

As principais relações identificadas no modelo são:

- USUARIO (1:0..1) MORADOR, com id\_usuario único em MORADOR.
- MORADOR (1:N) RESERVA.
- AREA\_COMUM (1:N) RESERVA.

A imagem 1 que se tem a seguir demonstra a estrutura do banco de dados em um Diagrama Entidade-Relacionamento com suas ligações e relacionamentos entre si, onde propõe ao usuário conseguir analisar de qual forma está configurado as interações de dados dentro do sistema.

Imagem 1 – Diagrama Entidade-Relacionamento



Fonte: Autoria própria

### 5.3 Integridade referencial e restrições

As tabelas utilizam chaves primárias INT IDENTITY. As chaves estrangeiras protegem os relacionamentos. UNIQUE é utilizado para e-mail, CPF, nome da área e vínculo do usuário do morador. CHECK controla tipo, status, capacidade e a ordem dos horários. Regras de negócio como conflito, área ativa, data futura e horário de funcionamento são validadas pelo back-end.

#### 5.3.1 Política de preservação dos registros

O sistema utiliza a inativação dos registros como forma de preservar o histórico das informações armazenadas no banco de dados. Dessa forma, moradores, áreas comuns e reservas que não estejam mais em utilização não são excluídos fisicamente quando a exclusão puder comprometer registros relacionados ou o histórico do sistema.

No caso das áreas comuns, por exemplo, quando uma área deixa de estar disponível, seu status é alterado de **ATIVA** para **INATIVA**, mantendo seus dados e possíveis reservas anteriores armazenados no banco. O mesmo princípio é aplicado aos usuários, que podem possuir status **ATIVO** ou **INATIVO**.

Essa abordagem permite que os registros permaneçam disponíveis para consultas futuras, auditorias e manutenção do histórico. Quando o sistema precisar apresentar somente os registros atualmente em utilização, as consultas podem utilizar uma condição sobre o campo de status, como WHERE status = 'ATIVO' ou WHERE status = 'ATIVA', conforme a tabela consultada.

Para as reservas, o cancelamento também é realizado de forma lógica, alterando seu status para **CANCELADA**, em vez de remover o registro do banco. Assim, o sistema mantém o histórico das reservas realizadas e permite diferenciar registros ativos de registros que já foram cancelados.

Essa decisão contribui para a integridade referencial do banco de dados, evita a perda desnecessária de informações e mantém o histórico das operações realizadas no sistema.

## 6 ARQUITETURA DE SOFTWARE E BACK-END

O navegador envia requisições HTTP ao servidor Python. O módulo de rotas direciona cada operação aos módulos de negócio, que realizam validações e utilizam o módulo de banco para executar SQL no SQL Server.

Fluxo: Front-end → servidor.py → rotas.py → módulos de negócio → banco.py → SQL Server.

### 6.1 Atribuições dos módulos

- servidor.py: servidor HTTP.
- rotas.py: rotas GET e POST.
- banco.py: conexão e operações SQL.
- usuario.py: login e usuário.
- morador.py: validações e operações de moradores.
- area.py: validações e operações de áreas.
- reserva.py: regras de data, horário, área e conflito.

## **7 IMPLEMENTAÇÃO E ACESSO A DADOS**

Foi realizada a implementação do banco de dados e das funcionalidades responsáveis pelo armazenamento, consulta e gerenciamento das informações do sistema. O banco de dados foi desenvolvido em SQL Server, utilizando comandos SQL para criação das tabelas, definição das chaves, relacionamentos e restrições de integridade. A implementação possui tabelas responsáveis pelo cadastro de usuários, moradores e áreas comuns do condomínio, permitindo organizar as informações de forma estruturada e facilitar o acesso aos dados pelo sistema.

### **7.1 Autenticação e gerenciamento de moradores**

O login consulta o usuário no banco, identifica seu tipo e verifica seu status. Usuários inativos são impedidos de prosseguir. O cadastro de morador utiliza transação para criar USUARIO e MORADOR, realizando rollback em caso de erro.

### **7.2 Lógica de conflito de horários**

Antes da inserção, o sistema consulta reservas ATIVAS para a mesma área e data e verifica sobreposição de horários. A lógica permite horários consecutivos. Também são validados área ativa, data futura, intervalo de 08:00 a 23:00 e horário final posterior ao inicial.

## 8 CAMADA WEB E INTERFACE

A interface foi construída com HTML5, CSS3 e JavaScript puro. O layout utiliza formulários, tabelas, cards, modais, menus laterais e media queries para adaptação a diferentes tamanhos de tela.

### 8.1 Rotas da aplicação

- /login
- /moradores/cadastrar
- /moradores/consultar
- /moradores/alterar
- /areas/cadastrar
- /areas/consultar
- /areas/alternar\_status
- /reservas/criar
- /reservas/consultar
- /reservas/cancelar

### 8.2 Interface e lógica do cliente

O JavaScript utiliza Fetch API para comunicação com o servidor. Informações de sessão necessárias à navegação são mantidas no localStorage. Os dados principais permanecem persistidos no SQL Server. Há validações de campos e formatação de CPF, telefone e apartamento, além de mensagens de erro e modais.

## 9 TESTES E DADOS DE TESTE

Foram preparados dados para verificar usuários ativos e inativos, áreas ativas e inativa, reservas ativas e canceladas e situações de horários consecutivos. O cenário de horários consecutivos deve permanecer dentro do funcionamento definido para o sistema, de 08:00 a 23:00.

Na massa de testes, o cenário consecutivo foi ajustado para João: 18:00–22:00 e Maria: 22:00–23:00, preservando a demonstração de que o término de uma reserva pode coincidir com o início da seguinte.

As evidências da criação do banco de dados e da inserção dos dados utilizados nos testes encontram-se no **Apêndice C**, por meio de registros visuais das etapas realizadas no SQL Server.

As evidências do sistema web encontram-se no **Apêndice D**.

## 10 INSTALAÇÃO E EXECUÇÃO

### Requisitos de instalação

- Git – utilizado para baixar o projeto do GitHub.
- Python – utilizado para executar o backend.
- Microsoft SQL Server – utilizado para armazenar os dados do sistema.
- SQL Server Management Studio (SSMS) – utilizado para criar, consultar e administrar o banco de dados.
- Driver ODBC do SQL Server – utilizado pelo Python por meio do pyodbc para realizar a conexão com o banco.

### Instalação do Git

- Baixe e instale o Git no computador. Durante a instalação, as opções padrão podem ser mantidas.  
Depois da instalação, abra o CMD ou Git Bash e execute: `git --version`
- Se for exibida a versão instalada do Git, a instalação foi concluída corretamente.

### Instalação do Python

- Instale o Python 3.14.7 e, durante a instalação no Windows, marque a opção "Add Python.exe to PATH".
- Depois, verifique a instalação com:
- `python --version`  
ou:
- `py --version`

### Instalação do SQL Server e do SSMS

- Instale o Microsoft SQL Server e, separadamente, o SQL Server Management Studio (SSMS). O SQL Server será responsável pelo armazenamento dos dados, enquanto o SSMS será utilizado para executar os scripts SQL e administrar o banco.
- Durante a configuração, anote o nome da instância do SQL Server, pois ele será necessário na configuração da conexão do Python. Exemplos comuns são:
- localhost

- .\SQLEXPRESS
- NOME-DO-PC\SQLEXPRESS

### **Baixar o projeto do GitHub**

- Abra o CMD, escolha uma pasta para armazenar o projeto e execute:
- `cd C:\Projetos`
- `git clone https://github.com/marcosdiasss/Sistema-de-Reservas-Condominio.git`
- Depois, entre na pasta do projeto:
- `cd Sistema-de-Reservas-Condominio`

### **Estrutura principal do projeto**

- O repositório apresenta, entre outras, as seguintes pastas principais:
- Backend
- Frontend
- `condominio_np1_v2`
- Para a execução do sistema, as pastas Backend e Frontend são as principais referências.

### **Configuração do banco de dados**

- Abra o SQL Server Management Studio e conecte-se à instância do SQL Server instalada. O banco utilizado pelo sistema é denominado:
- `trabalhonp1`
- Execute o script SQL disponibilizado no projeto para criar o banco, as tabelas, as restrições e os dados de teste. Ao final, devem existir as tabelas principais:
- `USUARIO`
- `MORADOR`
- `AREA_COMUM`
- `RESERVA`

### **Instalação do pyodbc**

- O backend Python utiliza o pyodbc para realizar a comunicação com o SQL Server. No CMD, execute:
- `pip install pyodbc`

- Caso necessário, utilize:
- `python -m pip install pyodbc`
- Para verificar a instalação:
- `python -c "import pyodbc; print('pyodbc OK')"`

### **Driver ODBC do SQL Server**

- Verifique no Windows se existe um driver ODBC instalado para o SQL Server. Procure por "Fontes de Dados ODBC" ou "ODBC Data Sources". É comum encontrar versões como ODBC Driver 17 for SQL Server ou ODBC Driver 18 for SQL Server.
- O nome do driver configurado no arquivo de conexão do projeto deve corresponder ao driver instalado no computador.

### **Configuração da conexão Python → SQL Server**

- A conexão deve informar o driver, o servidor/instância e o banco trabalhomp1. Quando o SQL Server estiver configurado para autenticação do Windows, uma conexão pode seguir este modelo:

```
import pyodbc
```

```
SERVIDOR = "NOME DA SUA CONEXÃO LOCAL"
```

```
BANCO = "NOME DO SEU BANCO"
```

- Importante: este trecho é apenas um modelo. O nome do servidor e o número do driver devem ser ajustados conforme a instalação do computador e conforme a configuração existente no projeto.

### **Teste da conexão com o banco**

- Antes de executar o sistema completo, recomenda-se testar separadamente a conexão entre Python e SQL Server. Se o projeto possuir um arquivo ou rotina específica para teste de conexão, utilize-a. Uma conexão bem-sucedida deve indicar que o Python conseguiu acessar o banco trabalhomp1.

### **Execução do Backend**

- Entre na pasta Backend:
- `cd Backend`
- Execute o arquivo de inicialização definido pelo projeto. No sistema desenvolvido para o trabalho, o servidor Python utiliza a porta 8000. Portanto, após a inicialização, o serviço deverá estar disponível localmente em:
- `http://localhost:8000`
- O terminal que estiver executando o servidor deve permanecer aberto durante a utilização do sistema.

### **Execução do Frontend**

- O frontend é composto pelas páginas HTML, estilos CSS e scripts JavaScript. Ele realiza as requisições ao backend, que por sua vez acessa o SQL Server.
- Com o backend em execução, abra no navegador o endereço disponibilizado pelo servidor, conforme a configuração do projeto.
- Para acesso ao sistema é necessário o uso dos seguintes usuários já cadastrados: Administrador - E-mail: `admin@condominio.com` /Senha: `admin123`. Morador – E-mail: `morador@condominio.com` /Senha: `morador123`.

### **Fluxo de funcionamento**

- A comunicação entre as partes do sistema ocorre da seguinte forma:
- Navegador → Frontend (HTML/CSS/JavaScript) → Backend Python → pyodbc → SQL Server → banco trabalhonp1

### **Testes após a instalação**

Após iniciar o sistema, recomenda-se verificar as funções principais:

- Acesso à tela de login.
- Login de administrador.
- Login de morador.
- Consulta de moradores.
- Consulta de áreas comuns.
- Cadastro e alteração de áreas, quando permitido pelo perfil.
- Criação de uma reserva.

- Impedimento de reservas conflitantes.
- Cancelamento de reserva.
- Consulta do histórico de reservas.

### **Ordem recomendada para execução**

1. Instalar Git.
2. Instalar Python.
3. Instalar SQL Server LocalDB.
4. Instalar SQL Server Management Studio.
5. Instalar o driver ODBC do SQL Server.
6. Baixar o projeto do GitHub.
7. Criar/configurar o banco trabalhonp1.
8. Executar os scripts SQL do projeto.
9. Instalar o pacote pyodbc.
10. Configurar e testar a conexão Python → SQL Server.
11. Iniciar o backend.
12. Abrir o frontend no navegador.
13. Realizar os testes funcionais.

### **Solução de problemas comuns**

- "python não é reconhecido": Verifique se o Python foi instalado com a opção Add Python.exe to PATH ou utilize o comando py.
- Erro ao importar pyodbc: Execute pip install pyodbc ou python -m pip install pyodbc.
- Erro de conexão com SQL Server: Confira se o serviço do SQL Server está iniciado e se SERVER, banco, autenticação e driver estão corretos.
- Banco não encontrado: Confirme no SSMS se o banco trabalhonp1 foi criado e se o nome utilizado na conexão está correto.
- Erro de driver ODBC: Verifique se o driver indicado na string de conexão está realmente instalado no Windows.
- Página não abre em localhost:8000: Confirme se o backend foi iniciado e se o terminal do servidor continua aberto.

## **11 CONSIDERAÇÕES FINAIS**

O projeto do Condomínio Sun Lake apresenta uma aplicação web integrada a banco de dados relacional para gerenciamento de moradores, áreas comuns e reservas. As tecnologias utilizadas estão de acordo com as opções estabelecidas para a atividade.

A modelagem utiliza relacionamentos, chaves e restrições para manter a integridade. As validações do back-end complementam o banco ao tratar regras como conflito de horários, área ativa, datas e período de funcionamento.

**APÊNDICE A – DDL DO BANCO DE DADOS**

```
CREATE DATABASE trabalhonp1
```

```
GO
```

```
USE trabalhonp1
```

```
GO
```

```
CREATE TABLE USUARIO (
```

```
    id_usuario INT IDENTITY(1,1) PRIMARY KEY,
```

```
    email VARCHAR(100) NOT NULL UNIQUE,
```

```
    senha VARCHAR(255) NOT NULL,
```

```
    tipo_usuario VARCHAR(20) NOT NULL,
```

```
    status VARCHAR(20) NOT NULL,
```

```
    CONSTRAINT CK_USUARIO_TIPO CHECK (tipo_usuario IN  
( 'ADMINISTRADOR', 'MORADOR' )),
```

```
    CONSTRAINT CK_USUARIO_STATUS CHECK (status IN ('ATIVO', 'INATIVO'))  
);
```

```
GO
```

```
CREATE TABLE MORADOR (
```

```
    id_morador INT IDENTITY(1,1) PRIMARY KEY,
```

```
    id_usuario INT NOT NULL UNIQUE,
```

```
    nome VARCHAR(100) NOT NULL,
```

```
    cpf CHAR(11) NOT NULL UNIQUE,
```

```
    telefone VARCHAR(15) NOT NULL,
```

```
    bloco VARCHAR(10) NOT NULL,
```

```
    apartamento VARCHAR(10) NOT NULL,
```

```
    CONSTRAINT FK_MORADOR_USUARIO FOREIGN KEY (id_usuario)  
REFERENCES USUARIO(id_usuario)
```

```
);
```

```
GO
```

```
CREATE TABLE AREA_COMUM (
```

```

id_area INT IDENTITY(1,1) PRIMARY KEY,
nome VARCHAR(100) NOT NULL UNIQUE,
descricao VARCHAR(255),
capacidade INT NOT NULL,
status VARCHAR(20) NOT NULL,
CONSTRAINT CK_AREA_CAPACIDADE CHECK (capacidade > 0),
CONSTRAINT CK_AREA_STATUS CHECK (status IN ('ATIVA', 'INATIVA'))
);
GO

```

```

CREATE TABLE RESERVA (
    id_reserva INT IDENTITY(1,1) PRIMARY KEY,
    id_morador INT NOT NULL,
    id_area INT NOT NULL,
    data_reserva DATE NOT NULL,
    hora_inicio TIME NOT NULL,
    hora_fim TIME NOT NULL,
    status VARCHAR(20) NOT NULL,
    CONSTRAINT FK_RESERVA_MORADOR FOREIGN KEY (id_morador)
REFERENCES MORADOR(id_morador),
    CONSTRAINT FK_RESERVA_AREA FOREIGN KEY (id_area) REFERENCES
AREA_COMUM(id_area),
    CONSTRAINT CK_RESERVA_HORARIO CHECK (hora_fim > hora_inicio),
    CONSTRAINT CK_RESERVA_STATUS CHECK (status IN ('ATIVA',
'CANCELADA'))
);
GO

```

## APÊNDICE B – DADOS DE TESTE

USE trabalhonp1

GO

```
INSERT INTO USUARIO (email, senha, tipo_usuario, status)
VALUES ('admin@condominio.com', 'admin123', 'ADMINISTRADOR', 'ATIVO');
INSERT INTO USUARIO (email, senha, tipo_usuario, status)
VALUES ('joao.silva@email.com', '123456', 'MORADOR', 'ATIVO');
INSERT INTO USUARIO (email, senha, tipo_usuario, status)
VALUES ('maria.souza@email.com', '123456', 'MORADOR', 'ATIVO');
INSERT INTO USUARIO (email, senha, tipo_usuario, status)
VALUES ('carlos.pereira@email.com', '123456', 'MORADOR', 'INATIVO');
GO
```

```
INSERT INTO MORADOR (id_usuario, nome, cpf, telefone, bloco, apartamento)
VALUES (2, 'João Silva', '11122233344', '(11) 91111-1111', 'A', '101');
INSERT INTO MORADOR (id_usuario, nome, cpf, telefone, bloco, apartamento)
VALUES (3, 'Maria Souza', '22233344455', '(11) 92222-2222', 'B', '202');
INSERT INTO MORADOR (id_usuario, nome, cpf, telefone, bloco, apartamento)
VALUES (4, 'Carlos Pereira', '33344455566', '(11) 93333-3333', 'C', '303');
GO
```

```
INSERT INTO AREA_COMUM (nome, descricao, capacidade, status)
VALUES ('Salão de Festas', 'Salão para eventos e comemorações', 50, 'ATIVA');
INSERT INTO AREA_COMUM (nome, descricao, capacidade, status)
VALUES ('Churrasqueira', 'Área de churrasqueira coberta', 20, 'ATIVA');
INSERT INTO AREA_COMUM (nome, descricao, capacidade, status)
VALUES ('Quadra Poliesportiva', 'Quadra em reforma', 30, 'INATIVA');
GO
```

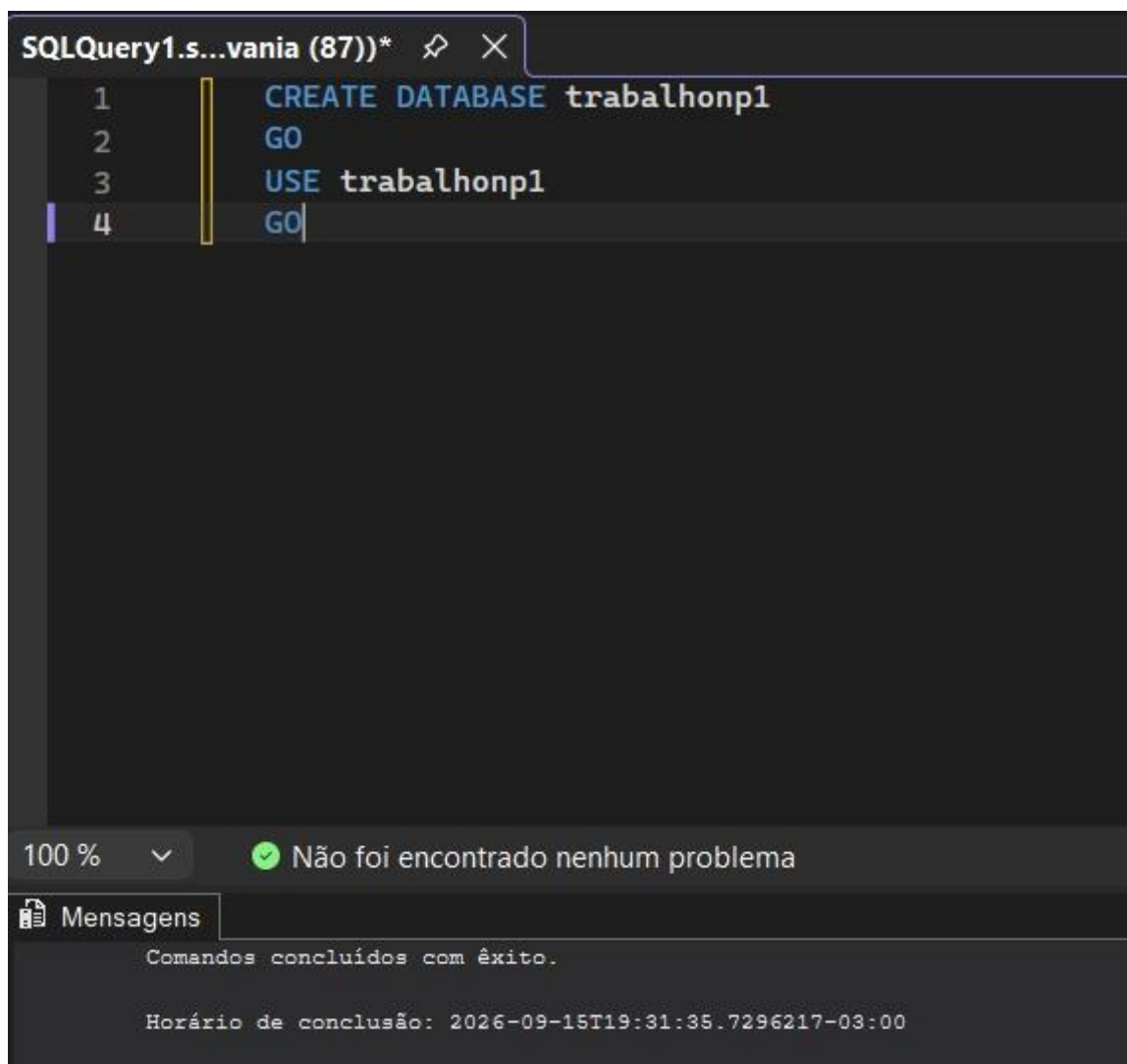
```
INSERT INTO RESERVA (id_morador, id_area, data_reserva, hora_inicio, hora_fim,
status)
```

```
VALUES (1, 1, '2026-09-20', '18:00', '22:00', 'ATIVA');  
INSERT INTO RESERVA (id_morador, id_area, data_reserva, hora_inicio, hora_fim,  
status)  
VALUES (2, 1, '2026-09-20', '22:00', '23:00', 'ATIVA');  
INSERT INTO RESERVA (id_morador, id_area, data_reserva, hora_inicio, hora_fim,  
status)  
VALUES (2, 2, '2026-09-21', '12:00', '16:00', 'ATIVA');  
INSERT INTO RESERVA (id_morador, id_area, data_reserva, hora_inicio, hora_fim,  
status)  
VALUES (1, 2, '2026-09-22', '10:00', '14:00', 'CANCELADA');  
GO
```

## APÊNDICE C – EVIDÊNCIAS VISUAIS DO BANCO

A imagem 2 a seguir representa a criação do banco de dados, responsável pelo armazenamento das informações do sistema.

Imagem 2 – Criação do banco de dados



```
SQLQuery1.s...vania (87))* ✕
1 CREATE DATABASE trabalhonp1
2 GO
3 USE trabalhonp1
4 GO
```

100 % ✓ Não foi encontrado nenhum problema

Mensagens

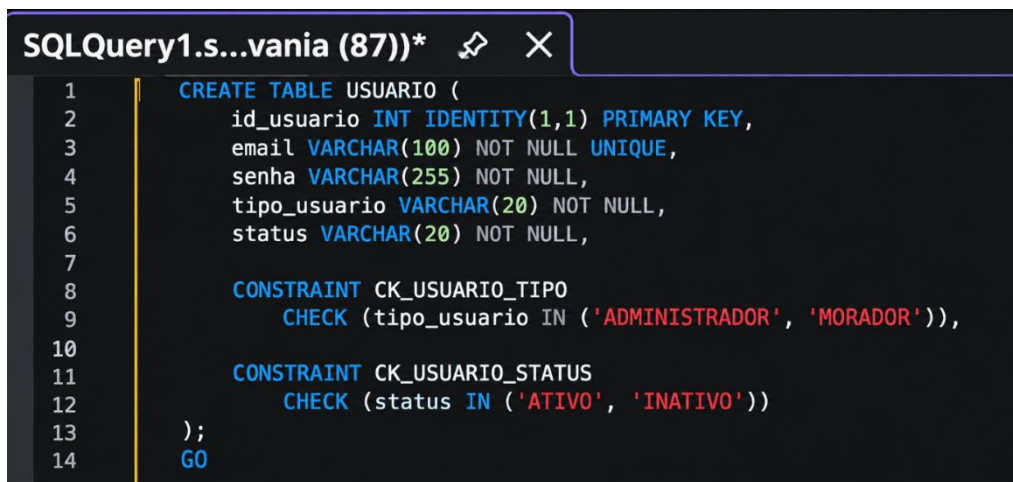
Comandos concluídos com êxito.

Horário de conclusão: 2026-09-15T19:31:35.7296217-03:00

Fonte: Autoria própria

As imagens 3, 4, 5 e 6 apresenta os comandos SQL utilizados para a criação e configuração das tabelas USUARIO, MORADOR, AREA\_COMUM e RESERVA do banco de dados, incluindo a definição dos campos, chaves e restrições necessárias para a organização e integridade das informações.

Imagem 3 – Criação da tabela USUARIO

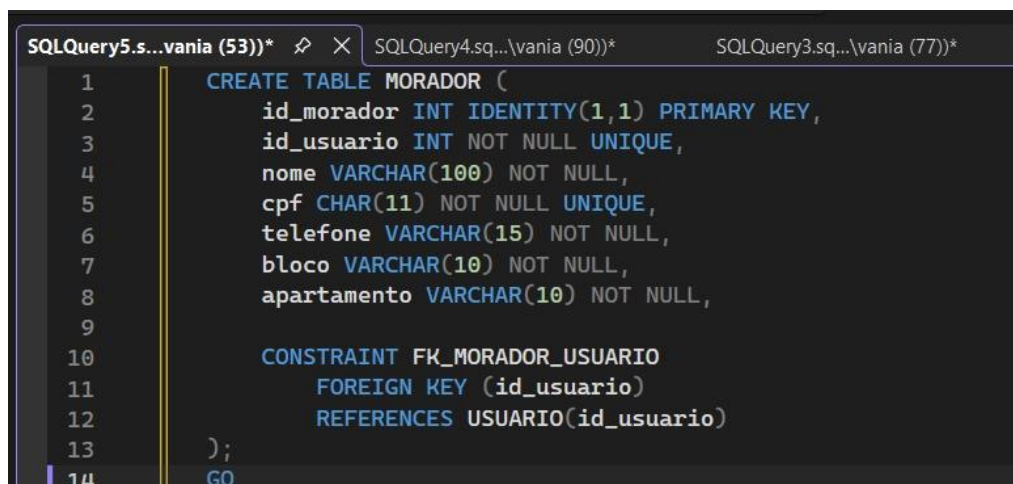


```

1  CREATE TABLE USUARIO (
2      id_usuario INT IDENTITY(1,1) PRIMARY KEY,
3      email VARCHAR(100) NOT NULL UNIQUE,
4      senha VARCHAR(255) NOT NULL,
5      tipo_usuario VARCHAR(20) NOT NULL,
6      status VARCHAR(20) NOT NULL,
7
8      CONSTRAINT CK_USUARIO_TIPO
9          CHECK (tipo_usuario IN ('ADMINISTRADOR', 'MORADOR')),
10
11     CONSTRAINT CK_USUARIO_STATUS
12         CHECK (status IN ('ATIVO', 'INATIVO'))
13 );
14 GO
  
```

Fonte: Autoria própria

Imagem 4 – Criação da tabela MORADOR

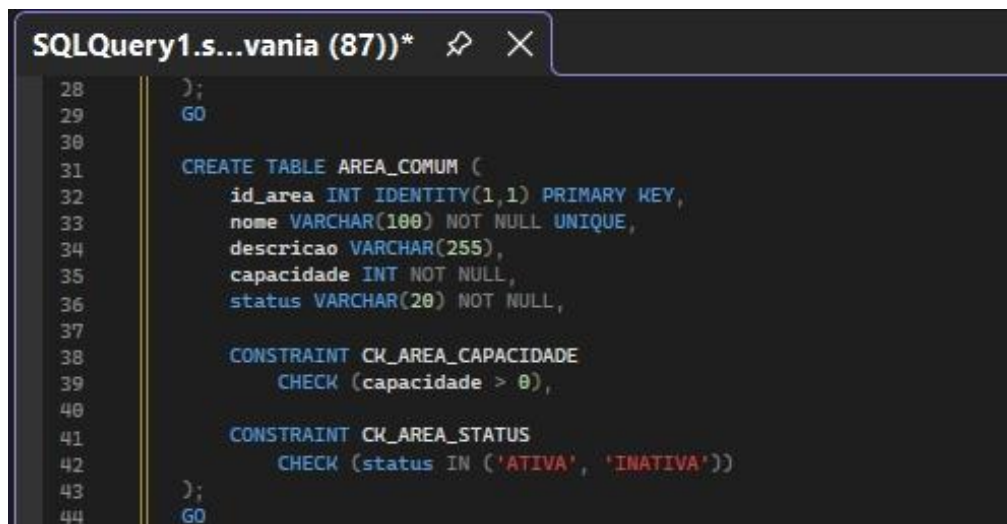


```

1  CREATE TABLE MORADOR (
2      id_morador INT IDENTITY(1,1) PRIMARY KEY,
3      id_usuario INT NOT NULL UNIQUE,
4      nome VARCHAR(100) NOT NULL,
5      cpf CHAR(11) NOT NULL UNIQUE,
6      telefone VARCHAR(15) NOT NULL,
7      bloco VARCHAR(10) NOT NULL,
8      apartamento VARCHAR(10) NOT NULL,
9
10     CONSTRAINT FK_MORADOR_USUARIO
11         FOREIGN KEY (id_usuario)
12         REFERENCES USUARIO(id_usuario)
13 );
14 GO
  
```

Fonte: Autoria própria

Imagem 5 – Criação da tabela AREA\_COMUM



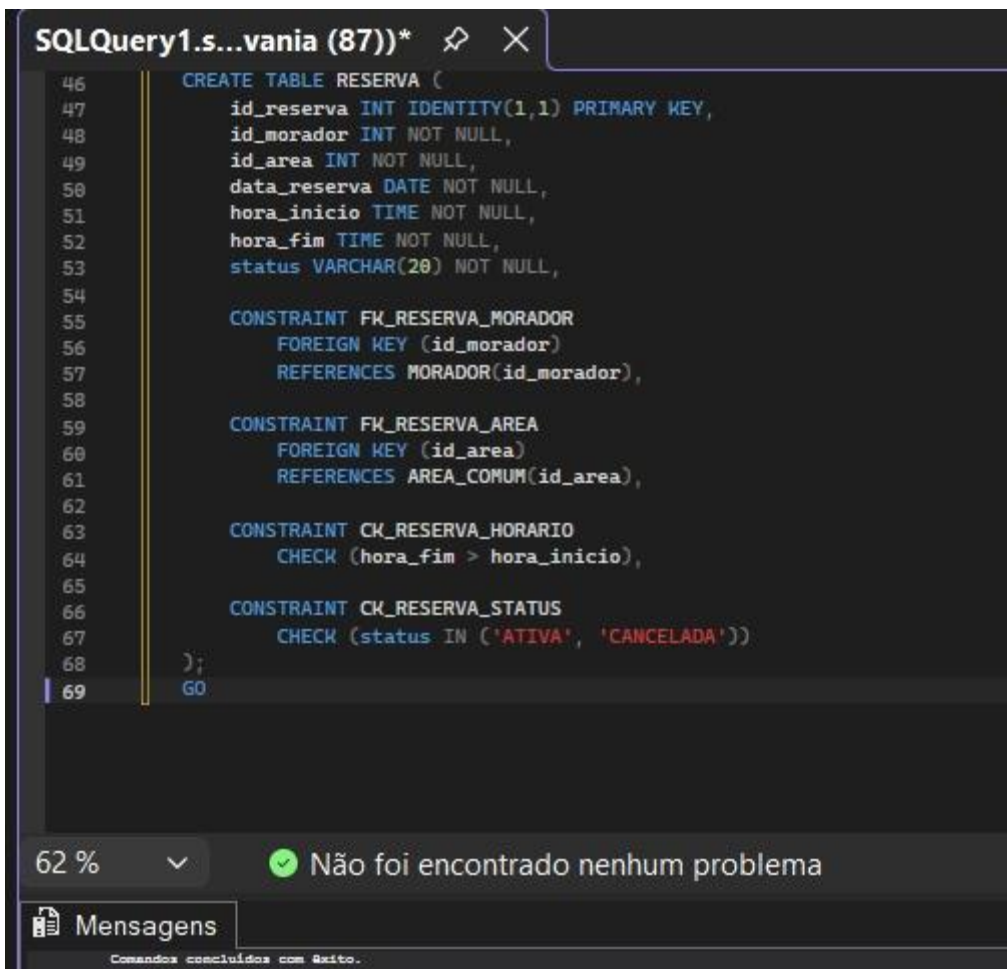
```

28      );
29      GO
30
31      CREATE TABLE AREA_COMUM (
32          id_area INT IDENTITY(1,1) PRIMARY KEY,
33          nome VARCHAR(100) NOT NULL UNIQUE,
34          descricao VARCHAR(255),
35          capacidade INT NOT NULL,
36          status VARCHAR(20) NOT NULL,
37
38          CONSTRAINT CK_AREA_CAPACIDADE
39              CHECK (capacidade > 0),
40
41          CONSTRAINT CK_AREA_STATUS
42              CHECK (status IN ('ATIVA', 'INATIVA'))
43      );
44      GO

```

Fonte: Autoria própria

Imagem 6 – Criação da tabela RESERVA



```

46      CREATE TABLE RESERVA (
47          id_reserva INT IDENTITY(1,1) PRIMARY KEY,
48          id_morador INT NOT NULL,
49          id_area INT NOT NULL,
50          data_reserva DATE NOT NULL,
51          hora_inicio TIME NOT NULL,
52          hora_fim TIME NOT NULL,
53          status VARCHAR(20) NOT NULL,
54
55          CONSTRAINT FK_RESERVA_MORADOR
56              FOREIGN KEY (id_morador)
57              REFERENCES MORADOR(id_morador),
58
59          CONSTRAINT FK_RESERVA_AREA
60              FOREIGN KEY (id_area)
61              REFERENCES AREA_COMUM(id_area),
62
63          CONSTRAINT CK_RESERVA_HORARIO
64              CHECK (hora_fim > hora_inicio),
65
66          CONSTRAINT CK_RESERVA_STATUS
67              CHECK (status IN ('ATIVA', 'CANCELADA'))
68      );
69      GO

```

62 %  Não foi encontrado nenhum problema

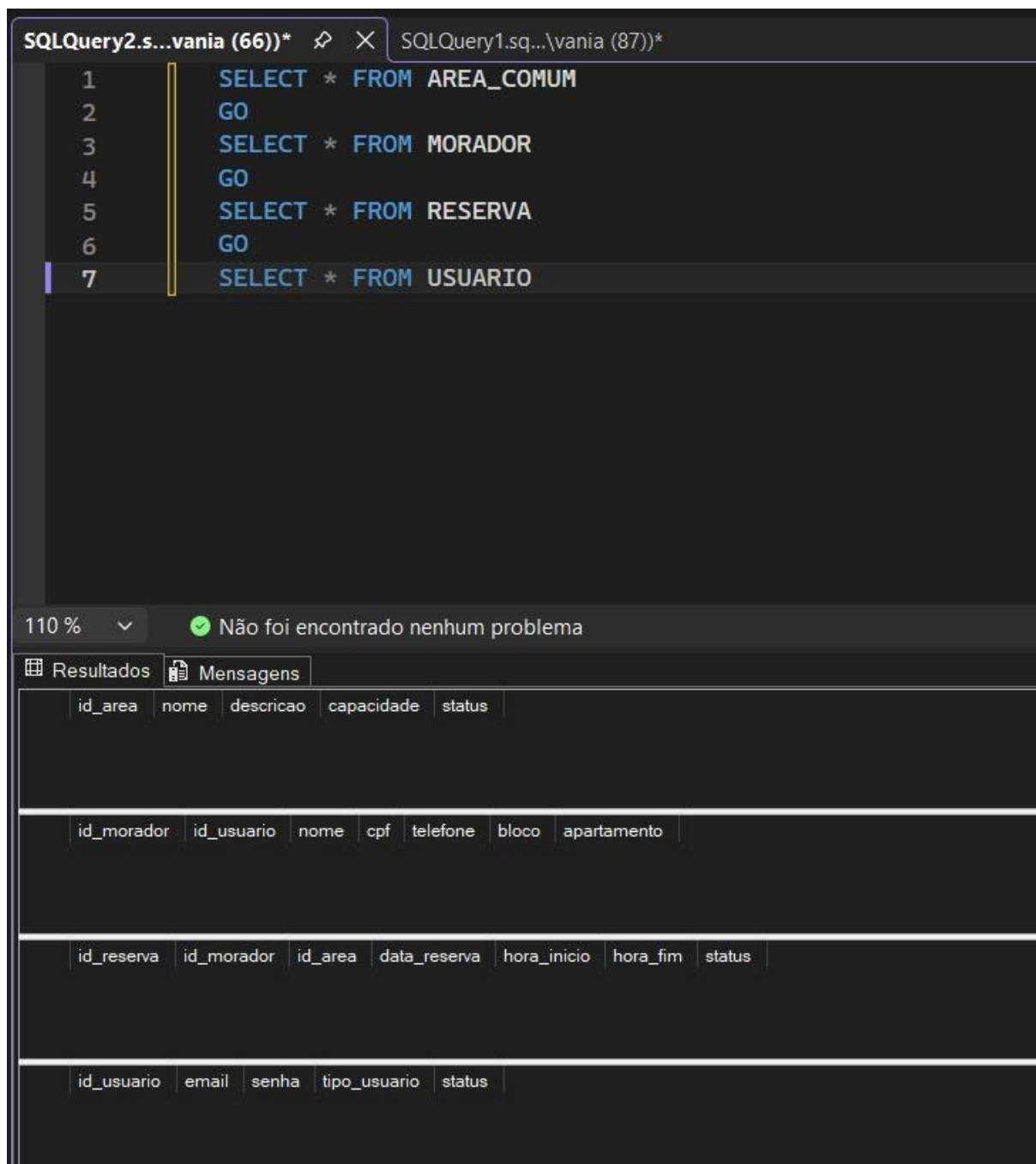
 Mensagens

Comandos concluídos com êxito.

Fonte: Autoria própria

A imagem 7 mostra a realização de consultas aos dados armazenados no banco de dados para verificar se as tabelas foram criadas corretamente.

Imagem 7 – Visualização de dados



Fonte: Autoria própria

A imagem 8 que se demonstra a inserção dos dados dos usuários no banco de dados.

Imagem 8 – Inserção de dados dos usuários

```
INSERT INTO USUARIO (email, senha, tipo_usuario, status)
VALUES ('admin@condominio.com', 'admin123', 'ADMINISTRADOR', 'ATIVO');

INSERT INTO USUARIO (email, senha, tipo_usuario, status)
VALUES ('joao.silva@email.com', '123456', 'MORADOR', 'ATIVO');

INSERT INTO USUARIO (email, senha, tipo_usuario, status)
VALUES ('maria.souza@email.com', '123456', 'MORADOR', 'ATIVO');

INSERT INTO USUARIO (email, senha, tipo_usuario, status)
VALUES ('carlos.pereira@email.com', '123456', 'MORADOR', 'INATIVO');
```

Fonte: Autoria própria

A imagem 9 mostra a inserção de dados na tabela dos moradores do condomínio.

Imagem 9 – Adição de moradores

```
1  USE trabalhonp1;
2  GO
3
4  INSERT INTO MORADOR
5  (id_usuario, nome, cpf, telefone, bloco, apartamento)
6  VALUES
7  (2, 'João Silva', '11122233344', '(11) 91111-1111', 'A', '101');
8
9  INSERT INTO MORADOR
10 (id_usuario, nome, cpf, telefone, bloco, apartamento)
11 VALUES
12 (3, 'Maria Souza', '22233344455', '(11) 92222-2222', 'B', '202');
13
14 INSERT INTO MORADOR
15 (id_usuario, nome, cpf, telefone, bloco, apartamento)
16 VALUES
17 (4, 'Carlos Pereira', '33344455566', '(11) 93333-3333', 'C', '303');
18 GO
```

5 % ☐ Não foi encontrado nenhum problema

Mensagens

(1 linha afetada)

(1 linha afetada)

(1 linha afetada)

Horário da conclusão: 2026-09-15T21:01:42.5209269-03:00

Fonte: Autoria própria

A imagem 10 apresenta a inserção dos dados das áreas comuns na tabela AREA\_COMUM, incluindo nome, descrição, capacidade e status de funcionamento.

Imagem 10 – Inserção de dados na tabela AREA\_COMUM

```
INSERT INTO AREA_COMUM (nome, descricao, capacidade, status)
VALUES ('Salão de Festas', 'Salão para eventos e comemorações', 50, 'ATIVA');

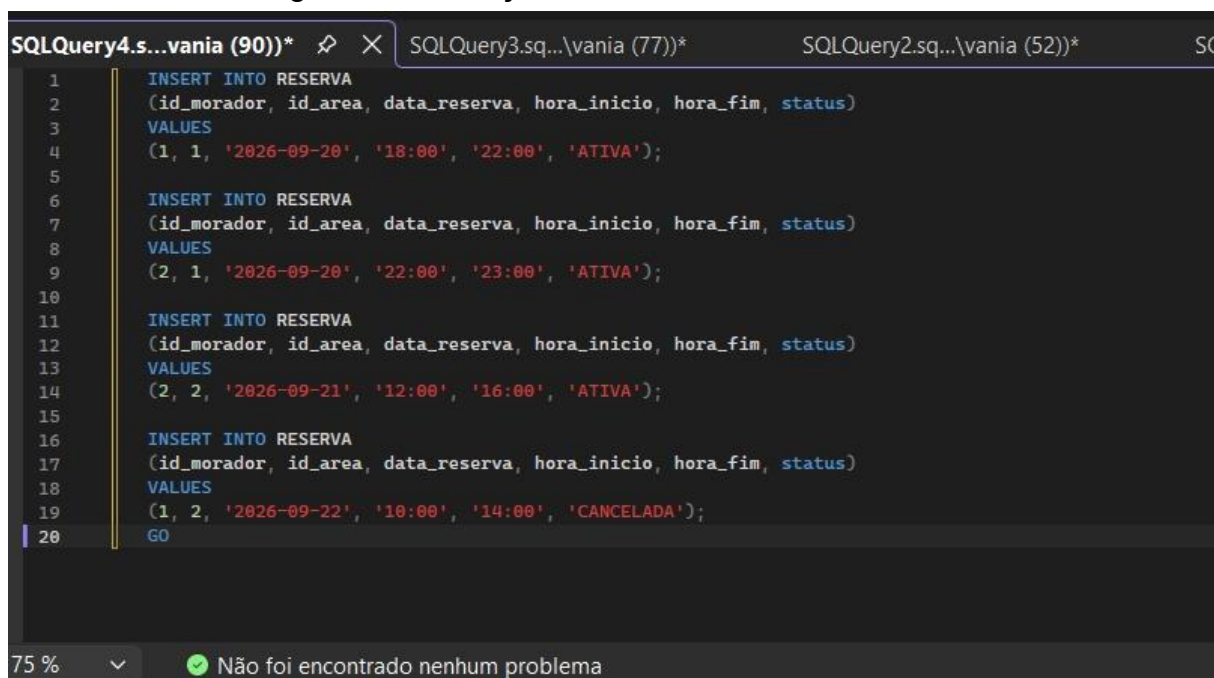
INSERT INTO AREA_COMUM (nome, descricao, capacidade, status)
VALUES ('Churrasqueira', 'Área de churrasqueira coberta', 20, 'ATIVA');

INSERT INTO AREA_COMUM (nome, descricao, capacidade, status)
VALUES ('Quadra Poliesportiva', 'Quadra em reforma', 30, 'INATIVA');
```

Fonte: Autoria própria

Em sequência tem-se a imagem 11, que demonstra a inserção dos dados das reservas.

Imagem 11 – Inserção de dados na tabela RESERVA



```
SQLQuery4.s...vania (90))*  SQLQuery3.sq...\vania (77))*  SQLQuery2.sq...\vania (52))*  SC
1  INSERT INTO RESERVA
2  (id_morador, id_area, data_reserva, hora_inicio, hora_fim, status)
3  VALUES
4  (1, 1, '2026-09-20', '18:00', '22:00', 'ATIVA');
5
6  INSERT INTO RESERVA
7  (id_morador, id_area, data_reserva, hora_inicio, hora_fim, status)
8  VALUES
9  (2, 1, '2026-09-20', '22:00', '23:00', 'ATIVA');
10
11  INSERT INTO RESERVA
12  (id_morador, id_area, data_reserva, hora_inicio, hora_fim, status)
13  VALUES
14  (2, 2, '2026-09-21', '12:00', '16:00', 'ATIVA');
15
16  INSERT INTO RESERVA
17  (id_morador, id_area, data_reserva, hora_inicio, hora_fim, status)
18  VALUES
19  (1, 2, '2026-09-22', '10:00', '14:00', 'CANCELADA');
20  GO

75 %  Não foi encontrado nenhum problema
```

Fonte: Autoria própria

Para finalizar temos a imagem 12 mostrando como ficaram as tabelas depois da inserção de dados.

Imagem 12 – SELECT para verificar os dados das tabelas

The screenshot shows a SQL IDE with the following queries and results:

```

1 SELECT * FROM AREA_COMUM
2 GO
3 SELECT * FROM MORADOR
4 GO
5 SELECT * FROM RESERVA
6 GO
7 SELECT * FROM USUARIO
  
```

110 % Não foi encontrado nenhum problema

**Resultados**

| 1 | 1 | Salão de Festas      | Salão para eventos e comemorações | 50 | ATIVA    |
|---|---|----------------------|-----------------------------------|----|----------|
| 2 | 2 | Churrasqueira        | Área de churrasqueira coberta     | 20 | ATIVA    |
| 3 | 3 | Quadra Poliesportiva | Quadra em reforma                 | 30 | INATI... |

|   | id_morador | id_usuario | nome           | cpf         | telefone        | bloco | apartamento |
|---|------------|------------|----------------|-------------|-----------------|-------|-------------|
| 1 | 1          | 2          | João Silva     | 11122233344 | (11) 91111-1111 | A     | 101         |
| 2 | 2          | 3          | Maria Souza    | 22233344455 | (11) 92222-2222 | B     | 202         |
| 3 | 3          | 4          | Carlos Pereira | 33344455566 | (11) 93333-3333 | C     | 303         |

|   | id_reserva | id_morador | id_area | data_reserva | hora_inicio        | hora_fim           | status     |
|---|------------|------------|---------|--------------|--------------------|--------------------|------------|
| 1 | 13         | 1          | 1       | 2026-09-20   | 18:00:00.000000... | 22:00:00.000000... | ATIVA      |
| 2 | 14         | 2          | 1       | 2026-09-20   | 22:00:00.000000... | 23:00:00.000000... | ATIVA      |
| 3 | 15         | 2          | 2       | 2026-09-21   | 12:00:00.000000... | 16:00:00.000000... | ATIVA      |
| 4 | 16         | 1          | 2       | 2026-09-22   | 10:00:00.000000... | 14:00:00.000000... | CANCELA... |

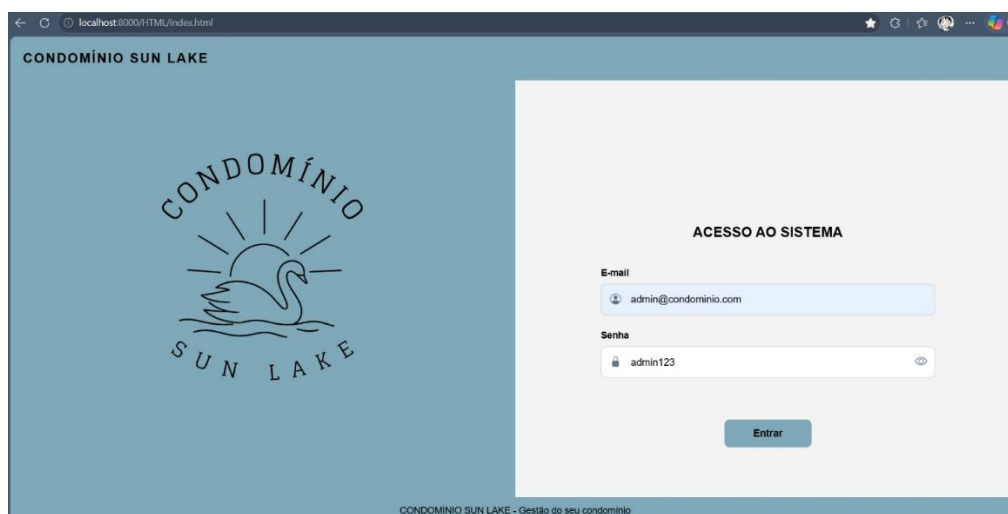
|   | id_usuario | email                    | senha    | tipo_usuario  | status |
|---|------------|--------------------------|----------|---------------|--------|
| 1 | 1          | admin@condominio.com     | admin123 | ADMINISTRADOR | ATI... |
| 2 | 2          | joao.silva@email.com     | 123456   | MORADOR       | ATI... |
| 3 | 3          | maria.souza@email.com    | 123456   | MORADOR       | ATI... |
| 4 | 4          | carlos.pereira@email.com | 123456   | MORADOR       | INA... |

Fonte: Autoria própria

## APÊNDICE D – EVIDÊNCIAS VISUAIS DO SISTEMA WEB

A primeira tela visual do sistema é a tela de login como representado na imagem 13, nessa etapa o usuário precisa preencher suas informações de e-mail e senha para poder seguir com as funcionalidades existentes, assim garante a segurança pois apenas pessoas cadastradas e autorizadas podem acessar o conteúdo do programa.

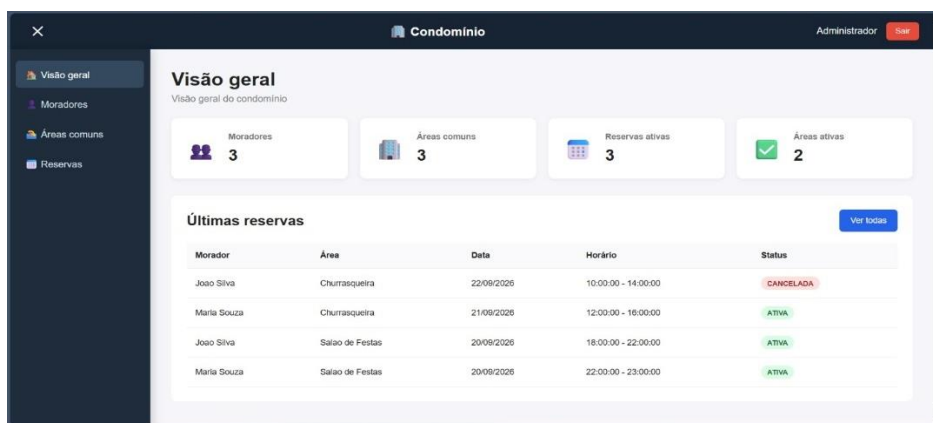
Imagem 13 – Tela de login



Fonte: Autoria própria

Em sequência, tem-se a tela inicial descrita como visão geral apresentado na imagem 14, o usuário pode visualizar quantos moradores o sistema possui cadastrado, tal como as áreas comuns, reservas ativas e as áreas comuns disponíveis, além de possui um menu para navegação dentro do sistema.

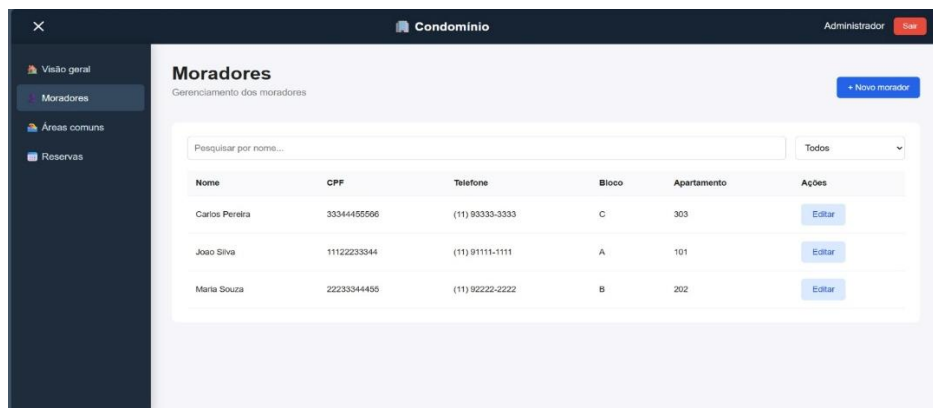
Imagem 14 – Tela inicial



Fonte: Autoria própria

Em sequência, tem-se a tela de gerenciamento dos moradores, apresentada na imagem 15, na qual o usuário pode visualizar os moradores cadastrados no sistema, contendo informações como nome, CPF, telefone, bloco e apartamento. A tela também disponibiliza a opção de pesquisar moradores pelo nome, filtrar os registros e realizar o cadastro de um novo morador, além da opção de editar os dados dos moradores já cadastrados.

Imagem 15 – Visualização de moradores



Fonte: Autoria própria

A seguir, tem-se a tela de edição de morador, apresentada na imagem 16, na qual o usuário pode alterar os dados cadastrais do morador selecionado, como nome completo, CPF, telefone, bloco e apartamento. A tela também disponibiliza as opções de cancelar a operação ou salvar as alterações realizadas.

Imagem 16 – Edição de cadastro

Fonte: Autoria própria

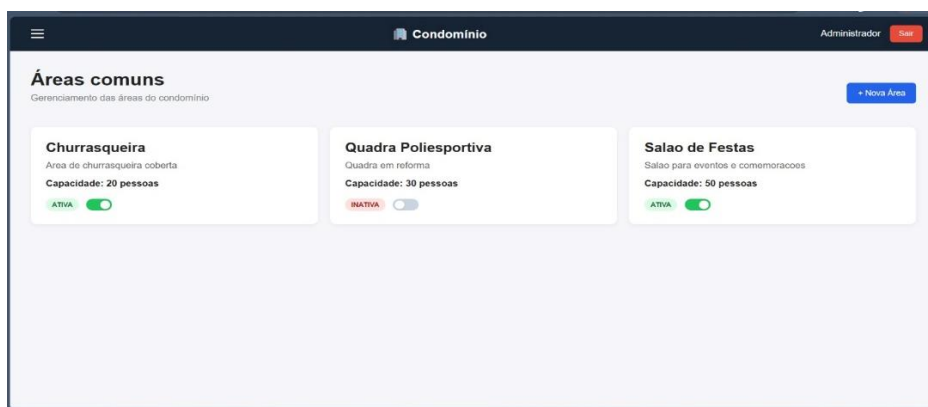
A tela de cadastro de um novo morador, apresentada na imagem 17, permite ao administrador inserir as informações necessárias para o registro no sistema. São apresentados campos para preenchimento do e-mail, senha, nome completo, CPF, telefone, bloco e apartamento. Após informar os dados, o administrador pode cancelar a operação ou finalizar o cadastro por meio do botão “Cadastrar Morador”.

Imagem 17 – Cadastro de morador

Fonte: Autoria própria

A tela de gerenciamento das áreas comuns, apresentada na imagem 18, permite ao administrador visualizar os espaços disponíveis no condomínio, com informações sobre sua descrição, capacidade e situação atual. No exemplo apresentado, são exibidas a churrasqueira, a quadra poliesportiva e o salão de festas, sendo possível identificar quais áreas estão ativas ou inativas. A tela também possui a opção de cadastrar uma nova área comum.

Imagem 18 - Gerenciamento



Fonte: Autoria própria

A tela de gerenciamento de reservas, apresentada na imagem 19, permite ao administrador acompanhar as reservas realizadas nas áreas comuns do condomínio. São exibidas informações como o morador responsável, a área reservada, a data, os horários de início e término e o status da reserva. Para as reservas que estão ativas, também é disponibilizada a opção de cancelamento, além do botão para realizar uma nova reserva.

Imagem 19 - Reservas

| Morador     | Área            | Data       | Início   | Fim      | Status    | Ações    |
|-------------|-----------------|------------|----------|----------|-----------|----------|
| Joao Silva  | Churrasqueira   | 22/09/2026 | 10:00:00 | 14:00:00 | CANCELADA |          |
| Maria Souza | Churrasqueira   | 21/09/2026 | 12:00:00 | 16:00:00 | ATIVA     | Cancelar |
| Joao Silva  | Salao de Festas | 20/09/2026 | 18:00:00 | 22:00:00 | ATIVA     | Cancelar |
| Maria Souza | Salao de Festas | 20/09/2026 | 22:00:00 | 23:00:00 | ATIVA     | Cancelar |

Fonte: Autoria própria

A tela de cadastro de uma nova reserva, apresentada na imagem 20, permite selecionar o morador e a área comum que será utilizada, além de definir a data e os horários de início e término da reserva. Após o preenchimento das informações, o usuário pode cancelar a operação ou confirmar a reserva por meio do botão disponível na tela.

Imagem 20 – Cadastro de reserva

**Nova reserva**  
Reserve uma área comum

Morador  
Selecione o morador

Área comum  
Selecione a área

Data da reserva  
dd/mm/aaaa

Hora inicial  
--:--

Hora final  
--:--

Cancelar Confirmar Reserva

Fonte: Autoria própria

O pop-up apresentado na imagem 21 corresponde à confirmação de saída do sistema. Ao selecionar a opção “Sair”, o sistema exibe uma janela de confirmação com a mensagem “Deseja realmente sair do sistema?”, oferecendo ao usuário duas alternativas: “Cancelar”, que mantém o usuário conectado, e “Sair”, que confirma o encerramento da sessão.

Imagem 21 – Poup-up de confirmação de saída do sistema

**Visão geral**  
Visão geral do condomínio

Moradores 3 Áreas comuns 3 Reservas ativas 3 Áreas ativas 2

**Sair do sistema**  
Deseja realmente sair do sistema?

Cancelar Sair

**Últimas reservas**

| Morador     | Área           | Data       | Horário             | Status    |
|-------------|----------------|------------|---------------------|-----------|
| Joao Silva  | Churrasqueira  | 22/09/2026 | 10:00:00 - 14:00:00 | CANCELADA |
| Maria Souza | Churrasqueira  | 21/09/2026 | 12:00:00 - 16:00:00 | ATIVA     |
| Joao Silva  | Sala de Festas | 20/09/2026 | 18:00:00 - 22:00:00 | ATIVA     |
| Maria Souza | Sala de Festas | 20/09/2026 | 22:00:00 - 23:00:00 | ATIVA     |

Ver todas

Fonte: Autoria própria